Ipcha Mistabra

Institutionalized Contradiction as a Verification Primitive
for Epistemically Robust Text Reviews

Oliver Baer
nyxCore Systems Research Division
research@nyxcore.cloud

March 2026

Abstract

Reviews of security and compliance text artifacts are central to established frameworks such as BSI IT-Grundschutz and NIST SSDF, yet remain heterogeneous and error-prone in practice [**?**, **?**]. Modern LLM-assisted text analysis exacerbates this problem through sycophancy—the tendency to confirm user assumptions even when incorrect [**?**]—and the architectural conflation of generation and validation within a single model instance. Existing mitigations, including formal inspections [**?**], Constitutional AI [**?**], and multi-agent debate [**?**], address partial aspects but do not combine mandatory adversarial roles, auditable artifacts, evidence binding, and formal gate rules into a single implementable protocol.

We introduce Ipcha Mistabra, a trialectic verification protocol comprising three roles: a *Proponent* (thesis) extracts atomic claims and produces an initial assessment; an *Ipcha Agent* (antithesis) systematically constructs counter-hypotheses grounded in authority documents; an *Auditor* (synthesis) consolidates findings, enforces evidence binding, and renders a formal gate decision. We define the protocol's formal objects (Claim, Assumption, Evidence, Finding, Gate Rule), provide implementable pseudocode with specified subfunctions, and introduce the *Ipcha Score*—a divergence-based proxy indicator of semantic displacement between initial and validated output.

A pilot case study applying the protocol to a production design specification surfaced 3 critical, 5 high-severity, and 4 medium/low findings (12 total) at a cost of $0.20 and 2.8 minutes of wall-clock time. All critical and high-severity findings were subsequently resolved through architectural hardening. We additionally present a within-subject evaluation design with statistical analysis plan for future controlled studies.

Keywords: text review; adversarial verification; LLM sycophancy; structured falsification; security analysis; formal gate rules.

ACM CCS Concepts:

- Security and privacy → Software security engineering.
- Software and its engineering → Software verification and validation.
- Computing methodologies → Natural language processing; Artificial intelligence.

1 Introduction

Security-relevant defects frequently originate in textual artifacts—requirements, architecture documents, policies, and threat models—before they manifest in code. Frameworks such as BSI IT-Grundschutz and NIST SSDF prescribe repeatable, documented reviews with evidence trails [**?**, **?**], yet in practice these reviews remain inconsistent: reviewers tend toward plausibility

confirmation rather than systematic falsification, and LLM-assisted reviews can reinforce this tendency through sycophancy [**?**].

Current mitigations target training-time or inference-time behavioral alignment. These improve average model behavior but leave a deeper architectural problem unaddressed: the same system instance is often responsible for both proposing and implicitly validating its own reasoning. This conflation of generation and validation within a single agent creates a structural blind spot that alignment alone cannot resolve.

1.1 Research Gap

Several existing approaches address overlapping aspects of this problem. Formal inspection methods enforce structured review with role separation [**?**]; Constitutional AI introduces rule-based self-critique [**?**]; multi-agent debate generates adversarial counter-arguments to improve factuality [**?**]; and orchestration frameworks such as AutoGen [**?**] provide infrastructure for multi-agent pipelines. However, none of these approaches combine all of the following in a single implementable protocol:

(i) extraction of atomic claims from text artifacts,

(ii) mandatory adversarial contradiction as a distinct role with auditable outputs,

(iii) evidence binding between claims and external authority documents, and

(iv) a formal gate rule over findings with severity-weighted blocking.

Table 1 provides a systematic feature comparison. Note that orchestration frameworks (AutoGen, CrewAI, LangGraph) are complementary infrastructure layers: they define *how* agents communicate, not *what* must happen during a review. Ipcha Mistabra specifies the verification semantics that can be implemented on top of any such framework.

1.2 Contributions

1. **Protocol.** Ipcha Mistabra as a trialectic verification primitive for text artifacts, with formal definitions, pseudocode, and specified subfunctions.

2. **Ipcha Score.** A divergence-based metric of semantic displacement between review phases, with specified embedding model and documented failure modes (Section 4).

3. **Pilot validation.** A case study demonstrating protocol application on a production design specification, surfacing 12 findings across two LLM providers (Section 6).

4. **Evaluation design.** A within-subject experimental framework with statistical analysis plan for future controlled studies.

2 Related Work

2.1 Formal Inspection and Structured Review

Fagan inspection [**?**] introduced role-separated code and document review (author, moderator, inspector, reader) with defined phases (overview, preparation, meeting, rework, follow-up). The method demonstrated significant defect-detection improvement but is manual, synchronous, and does not formalize evidence binding or produce machine-auditable artifacts. Ipcha Mistabra inherits the principle of role separation but extends it with formal claim/evidence structures and automated gate rules.

Feature	Fagan	CAI	Debate	ATLAS	**IM Protocol**
Role separation	✓	–	partial	–	✓
Mandatory contradiction	partial	partial	✓	–	✓
Evidence binding	–	–	–	–	✓
Auditable finding artifacts	–	–	–	partial	✓
Formal gate rule	–	–	–	–	✓
Runtime (not training-time)	✓	–	✓	✓	✓
Authority grounding	–	partial	–	✓	✓
Domain-agnostic design	✓	✓	✓	–	✓*

Table 1: Feature comparison of related approaches. *Evaluated in the security domain; generality is a design goal, not yet empirically validated across domains (see Section 7.2).

2.2 Failure Modes of LLM-Assisted Text Analysis

Sycophancy denotes the tendency of models to confirm user opinions at the expense of truth [**?**]. Perez et al. showed that model-written evaluations can systematically discover such behavioral failures [**?**]. For safety-relevant text reviews, this is especially critical: "confirmation" can be mistakenly interpreted as "validation," creating a false sense of security.

2.3 Constitutional AI and Rule-Based Self-Critique

Constitutional AI (CAI) [**?**] introduces a set of principles against which a model critiques its own outputs. As a single-model approach in its default deployment, CAI does not structurally prevent correlated blind spots: the same instance generates and evaluates. CAI also lacks auditable, persistent finding artifacts and has no formal gate mechanism. SPARROW [**?**] extends this with human-authored rules and targeted red-teaming but remains a training-time intervention rather than a runtime verification protocol.

2.4 Multi-Agent Debate

Du et al. demonstrated that multi-agent debate can improve factuality and reasoning by forcing explicit counter-arguments [**?**]. However, debate-based setups typically lack: (i) an explicit audit role with synthesis authority, (ii) evidence binding against external sources, and (iii) a formal gate that blocks output based on finding severity. Ipcha Mistabra operationalizes adversarial pressure as a verifiable protocol—with defined roles, typed artifacts, and gate rules—rather than a prompt-based heuristic.

2.5 Adversarial Testing and Red-Teaming

Perez et al. [**?**] and the MITRE ATLAS framework provide taxonomies and tooling for adversarial testing of AI systems. These focus on model-level vulnerability discovery rather than text-artifact review. Ipcha Mistabra addresses a different surface: it reviews the *outputs* (documents, specifications) rather than testing the *model itself*.

2.6 Systematic Positioning

3 Method

3.1 Problem Formulation

Given a text artifact T (e.g., specification, policy) and context K (domain, threat model, norms/authority documents), the objective is to produce an auditable report R containing

findings F, evidence bindings, and a remediation plan, together with a gate decision $g \in \{0, 1\}$ (release/block).

3.2 Formal Definitions

Definition 1 (Claim). A *claim* c is an atomic, verifiable statement extracted from T (e.g., "data is encrypted in transit"). The set of all claims is $C = \{c_1, \ldots, c_n\}$.

Definition 2 (Assumption). An *assumption* a is a prerequisite for c that is not explicitly evidenced in T (e.g., "a valid certificate chain is in place"). $A = \{a_1, \ldots, a_m\}$.

Definition 3 (Evidence). *Evidence* e is a traceable attestation (text passage, standard section, test log, code/configuration proof) supporting or refuting a claim. $E = \{e_1, \ldots, e_k\}$.

Definition 4 (Finding). A finding is a tuple

$$f = (id, c, sev, rationale, E_f, remediation, status)$$

with evidence subset $E_f \subseteq E$, severity $sev \in \{\text{critical}, \text{high}, \text{medium}, \text{low}\}$, and status $\in \{\text{accepted}, \text{refuted}, \text{needs-}$

Definition 5 (Gate Rule). A gate rule $\mathcal{G}(F) \in \{0, 1\}$ decides whether release is permitted:

$$\mathcal{G}(F) = 1 \iff \nexists f \in F : \ (f.status \neq \text{refuted}) \ \wedge \ (f.sev \geq \text{high}).$$

3.3 What Makes This a Protocol, Not a Prompt Chain

The term "protocol" is used deliberately. Three structural properties distinguish Ipcha Mistabra from prompt-based heuristics:

1. **Mandatory execution.** Contradiction is not an optional step; the workflow engine enforces it as a required stage before gate evaluation.
2. **Persistent, auditable artifacts.** Each phase produces a typed output (O_p, O_i, O_f) that is stored, versioned, and available for external audit.
3. **Formal gate mechanism.** The gate rule $\mathcal{G}$ operates over structured finding objects, not free-text opinions; a BLOCK decision prevents release regardless of subjective confidence.

A prompt chain may implement similar reasoning steps, but without enforcement, persistence, and gating it remains advisory rather than binding.

3.4 Protocol Overview (Trialectic)

Ipcha Mistabra employs three roles (see Figure 1):

- **Proponent (P):** extracts claims and assumptions, produces initial review O_p and findings F_p.
- **Ipcha Agent (I):** generates counter-narratives, counter-hypotheses, and evidence demands; produces O_i and F_i.
- **Auditor (A):** consolidates, deduplicates, and prioritizes; enforces evidence binding; renders gate decision; produces final O_f.

3.5 Step-by-Step Protocol

Step 0: Scope Fixation. Define objective, artifact class, threat model, and authority documents (e.g., BSI/NIST references) [**?**, **?**].

Step 1: Claim Extraction. Segment T; extract C and A; normalize to atomic statements.

Step 2: Proponent Pass. Generate O_p and findings F_p with evidence requirements.

Step 3: Ipcha Pass (Contradiction). Falsify: counterexamples, contradictions, missing prerequisites, norm/policy conflicts; generate O_i, F_i.

Step 4: Audit Resolution. Merge $F = \text{merge}(F_p, F_i)$; render status decisions; produce remediation plan.

Step 5: Gate. Apply $\mathcal{G}$ (release/block).

Step 6: Metrics. Compute review metrics (finding counts, effort, Ipcha Score).

3.6 Top-Level Pseudocode

Algorithm 1 IPCHA MISTABRA: Institutionalized Contradiction for Text Artifact Review

Require: Text artifact T, context K, gate rule $\mathcal{G}$
Ensure: Report R, findings F, gate decision g

```
(C, A) ← ExtractClaims(T)                                  ▷ Algorithm 2
(O_p, F_p) ← ProponentReview(T, K, C, A)
(O_i, F_i) ← IpchaContradict(T, K, C, A, O_p, F_p)         ▷ Algorithm 3
F ← MergeAndNormalize(F_p, F_i)
(O_f, F) ← AuditAndResolve(T, K, O_p, O_i, F)
g ← G(F)
R ← AssembleReport(T, K, O_f, F, g)
return R, F, g
```

3.7 Subfunction Specifications

Algorithm 2 EXTRACTCLAIMS: Atomic Claim and Assumption Extraction

Require: Text artifact T
Ensure: Claims C, Assumptions A

```
S ← SentenceSegment(T)                                     ▷ Split T into sentence-level units
for each sentence s ∈ S do
    cands ← IdentifyVerifiable(s)                          ▷ Filter for testable propositions
    for each candidate v ∈ cands do
        c ← NormalizeToAtomic(v)                           ▷ One proposition per claim
        C ← C ∪ {c}
        A_c ← InferPrereqs(c, T)                           ▷ What must hold for c?
        A ← A ∪ A_c
    end for
end for
return C, A
```

When instantiated with an LLM, each subfunction maps to a structured prompt with typed output schema. Appendix A provides reference prompt templates.

Algorithm 3 IpchaContradict: Structured Falsification Pass

Require: T, K, C, A, O_p, F_p
Ensure: Contradictor output O_i, contradictor findings F_i

```
F_i ← ∅
for each claim c ∈ C do
    H⁻ ← CounterHypotheses(c, K)          ▷ Strongest counter-narratives
    V ← AuthorityCheck(c, K)              ▷ Norm/policy violations
    M ← MissingEvidence(c, O_p)           ▷ Gaps in Proponent reasoning
    if H⁻ ≠ ∅ or V ≠ ∅ or M ≠ ∅ then
        f_i ← BuildFinding(c, H⁻, V, M)
        F_i ← F_i ∪ {f_i}
    end if
end for
for each assumption a ∈ A do
    ChallengeAssumption(a, K)             ▷ Demand evidence or flag
end for
O_i ← Summarize(F_i)
return O_i, F_i
```

4 Ipcha Score

We define the **Ipcha Score** as a divergence-based proxy indicator of semantic displacement between the Proponent output and the final Auditor output:

$$IS \;=\; 1 - \cos\big(\text{embed}(A_p),\ \text{embed}(A_f)\big) \tag{1}$$

where A_p is the Proponent output, A_f is the final Auditor output, and embed$(\cdot)$ denotes a shared embedding function.

Embedding model specification. For reproducibility, we fix the reference embedding to `text-embedding-3-large` (OpenAI, 3072 dimensions) as the default instantiation. Any alternative (e.g., Sentence-T5, BGE-M3, Cohere Embed v3) is permissible provided it is declared and held constant within a study. The score is *not* comparable across different embedding models.

Interpretation (see Figure 2).

- **Low score** (< 0.15): the original output was already robust, or critique was weak.
- **Mid-range** (0.15–0.45): normal dialectical refinement; the expected operating range for useful adversarial review.
- **High score** (> 0.45): substantial correction pressure; important weaknesses were discovered and resolved.

These thresholds are preliminary; empirical calibration across artifact types and domains is required before operational use.

Known limitations. The score measures *semantic surface displacement*, not epistemic correction directly. Two failure modes must be acknowledged:

1. **False positive (high score, low correction).** If the Auditor paraphrases the Proponent output without changing substance, the score inflates. Mitigation: complement with structured finding-level diff (count of status changes in F).

2. **False negative (low score, high correction).** A single critical sentence change (e.g., "system is secure" → "system has unmitigated CVE") may barely shift the embedding centroid. Mitigation: compute a *finding-weighted* variant $IS_w = \sum_{f \in F_\Delta} w(f.sev)$ over findings that changed status between Proponent and Auditor pass.

We recommend reporting *IS* alongside finding-level metrics (recall, precision) and not using it as a sole decision criterion. Future work should validate *IS* against human expert ratings of correction strength (correlation study).

5 Evaluation Design

5.1 Research Questions

- **RQ1:** Does Ipcha Mistabra increase finding recall and precision relative to single-pass reviews?
- **RQ2:** Does Ipcha Mistabra reduce risk-weighted residual errors and policy violations?
- **RQ3:** What cost/benefit trade-offs arise (time, compute, human rework)?
- **RQ4:** Does the Ipcha Score correlate with human-judged correction strength?

5.2 Experimental Design

Design: Within-subject: each artifact is reviewed with both a baseline (single-pass) and Ipcha Mistabra.
Artifact classes: D_{spec}, D_{policy}, D_{threat}, $D_{\text{compliance}}$.
Ground truth: Expert panel or consensus labeling; inter-rater reliability is reported.

5.3 Metrics

Metric	Definition / Measurement
Finding Recall	Proportion of ground-truth findings discovered
Finding Precision	Proportion of reported findings confirmed by experts
Risk-Weighted Residual Error	Sum of severity for unresolved findings after gate
Policy/Compliance Violations	Rate of violations against defined rules/standards
Review Effort	Time (min), token consumption, rework (min)
Ipcha Score (*IS*)	Divergence measure per Equation 1
Finding-Weighted Score (IS_w)	Severity-weighted status-change count
IRR	Cohen's κ / Krippendorff's α

Table 2: Evaluation metrics.

5.4 Hypothesized Direction of Effect

Based on findings in multi-agent debate [**?**] and structured inspection [**?**], we hypothesize the following directional effects. These are *not empirical results*; they define the evaluation target structure for future controlled studies.

Metric	Hypothesized direction	Rationale
Error rate	↓ (substantial)	Contradiction surfaces hidden flaws
Compliance violations	↓ (substantial)	Authority grounding enforces standards
Human review effort	↓ (moderate)	Automated pre-screening reduces manual load
Output validity	↑	Audit resolution improves final quality

Table 3: Hypothesized directional effects. Magnitude to be determined empirically.

5.5 Statistical Analysis Plan

Paired comparisons: Wilcoxon signed-rank (nonparametric) or paired t-test (if normality is plausible), plus effect size (Cliff's delta / Cohen's d).
Mixed models: Logistic/mixed effects for violations; "artifact" as random effect.
Multiple testing: FDR control (Benjamini–Hochberg).
Power: A priori power analysis; depends on expected effect size from pilot observations.
Score validation (RQ4): Spearman rank correlation between *IS* and expert-rated correction strength.

6 Pilot Case Study: Persona Evaluation Specification

To validate the protocol beyond its formal definition, we applied Ipcha Mistabra to a production design specification: the *Persona Evaluation v2* spec for a multi-tenant AI platform. The specification proposed a comprehensive overhaul of persona quality evaluation, introducing structured persona profiles, a judge-based scoring architecture, adversarial exploitation tests, and an auto-derivation pipeline. The document was internally consistent and considered implementation-ready by its author.

6.1 Workflow Execution

The protocol was instantiated using a five-step pipeline:

1. **Prepare.** Extract core thesis, key claims, and implicit assumptions from the artifact.
2. **Adversarial Analysis.** Multi-provider fan-out: the enriched specification was distributed across provider-lens combinations, each performing structured falsification from a distinct analytical perspective.
3. **Synthesis.** Aggregate cross-model findings; distinguish unique findings, convergent findings, contradictions, and blind spots.
4. **Arbitration.** Render final verdict, resolving inter-provider contradictions.
5. **Results.** Produce structured finding objects with severity, category, and actionable suggestions; pause for human review.

Provider configuration. Two providers were used with four analytical lenses each (Table 4).

Analytical lenses. Each provider-lens combination examined the artifact from a distinct perspective: *Security & Attack Surface* (injection vectors, data exposure), *Scalability & Operational* (performance under load, distributed deployment), *Organizational & Process* (adoption barriers, developer incentives, Goodhart's Law), and *General* (logical consistency, completeness, unstated assumptions).

Provider	Model	Tokens	Cost	Calls
Google	gemini-2.5-pro	64,139	$0.198	8
Moonshot	kimi-k2-0711-preview	13,634	$0.000	4
Total		**77,773**	**$0.198**	**12**

Table 4: Provider metrics for the pilot workflow. Total wall-clock time: 2.8 min.

Authority grounding. Context injected into each step included cross-project architectural patterns, mandatory authority documents from a retrieval-augmented generation system, accumulated code patterns, and persistent workflow insights from prior review cycles.

6.2 Claim Extraction

The Prepare step extracted 6 key claims and 7 implicit assumptions from the specification. Example claims: "An automated pipeline using an LLM can reliably derive a structured profile from a custom persona's system prompt" and "An LLM is sufficiently capable to act as a reliable judge for scoring another LLM's output against a complex rubric."

6.3 Findings

Cross-model consensus findings. Six findings were independently identified by two or more provider-lens combinations (Table 5).

ID	Sev.	Finding	Corroborating lenses
CM-01	BLOCKER	LLM-as-Judge unreliable and gameable (Oracle Fallacy)	Gem/Sec, Gem/Gen, Kimi/Sec, Kimi/Gen, Kimi/Scl
CM-02	CRIT	Automated profile gen. is a systemic attack vector	Gem/Sec, Gem/Scl, Gem/Org, Gem/Gen, Kimi/Sec
CM-03	CRIT	In-memory cache: catastrophic scaling bottleneck	Gem/Scl, Gem/Gen, Kimi/Scl, Kimi/Org
CM-04	HIGH	16× cost/latency cripples adoption	Gem/Scl, Gem/Org, Kimi/Gen
CM-05	HIGH	Granular scoring creates perverse incentives	Gem/Org, Kimi/Org
CM-06	HIGH	Tailored-only tests create Maginot Line defense	Gem/Gen, Kimi/Gen

Table 5: Cross-model consensus findings from the Synthesis step.

Unique findings. Six additional findings were identified by exactly one provider-lens combination, representing the highest-value insights from the diverse redundancy approach:

- **UF-01** (CRITICAL, Gem/Sec): *Profile Generation Injection*—a user's system prompt can directly instruct the parser LLM to generate a deliberately weak test suite.
- **UF-02** (HIGH, Gem/Sec): *Rubric Injection*—a persona's output mimics the judge's rubric language, exploiting sycophantic pattern-matching.
- **UF-03** (HIGH, Kimi/Sec): *Test Case Exhaustion as Oracle Attack*—detailed scoring breakdowns enable iterative reverse-engineering of persona rules.
- **UF-04** (MEDIUM, Kimi/Sec): *Cache Poisoning via Timing Side-Channel*—the 24h TTL creates observable timing differences that leak profile structure.

- **UF-05** (MEDIUM, Kimi/Org): *Organizational Circuit Breaker*—automatic reversion to generic testing when evaluation overhead exceeds a threshold.
- **UF-06** (LOW, Kimi/Sec): *Zero-Knowledge Profile Generation*—cryptographic methods to prevent profile data from existing in readable form.

Philosophical contradiction. The analysis surfaced a fundamental design tension: *Position A* (Gemini): the profile's explicit structure is a strength; harden the processes around it. *Position B* (Kimi): the profile's machine-readable nature *is* the vulnerability; the solution is obfuscation. The hardened specification adopted Position A (transparency enables auditability), acknowledging Position B as a deferred design trade-off. This disagreement was not an analytical failure but one of the most valuable outputs of the protocol—transforming an implicit assumption into an explicit, documented design decision.

Blind spots. The Synthesis step identified four areas not covered by any provider: foundation model baseline shift, test data provenance, legal liability of the reasoning audit trail, and developer experience of debugging a stochastic judge.

Arbitration verdict. Adversarial robustness score: **25/100**. Verdict: "Conceptually sound, implementation fragile." Gate decision: $\mathcal{G}(F) = 0$ (BLOCK).

6.4 Hardening Applied

All critical and high findings were resolved in the specification revision:

Aspect	Before (original)	After (hardened)
Scoring	LLM judge sole arbiter	Deterministic primary, LLM advisory
Discrepancy	No detection	Flagged when determ./LLM diverge >30 pts
Profile gen.	Fully automated	Human-in-the-loop approval gate
Profile storage	In-memory Map (24h TTL)	DB-persisted with approval status
Jailbreak tests	Persona-specific only	Generic (3) + persona-specific (5)
Eval cost	22 calls (single tier)	Quick (∼10) / Full (∼22) tiered

Table 6: Before/after comparison of key architectural decisions.

6.5 Observations

Four observations from the pilot generalize beyond this specific artifact:

1. **Design specifications for AI evaluation systems are attack surface declarations.** Every interface, pipeline, and caching strategy encodes assumptions that can be exploited.
2. **LLM-generated components that constrain other LLM outputs create circular dependencies.** Auto-derivation pipelines collapse the security boundary: the LLM is simultaneously the subject of evaluation and the author of evaluation criteria. Human checkpoints break this circularity.
3. **Cross-model disagreement reveals design tensions, not analytical failures.** The transparency-vs-obfuscation contradiction was one of the protocol's most valuable outputs.
4. **The protocol's value scales with artifact complexity.** At a total cost of $0.20 and 2.8 minutes of wall-clock time, the analysis surfaced 3 critical (including 1 blocker), 5 high, and 4 medium/low findings—12 in total—that would have required multiple expert reviewers and substantially more time to identify manually.

7 Discussion

7.1 Institutionalized Contradiction as Systems Primitive

The IM Protocol reframes AI reliability as a systems problem rather than a pure model-alignment problem. The approach is *inspired by* fault tolerance in distributed systems, where robustness derives from structured checks between components rather than trust in any single component. The analogy is structural, not formal: the IM Protocol does not provide Byzantine fault tolerance or provable safety/liveness guarantees. Its contribution is to make adversarial challenge a mandatory, logged, and enforceable stage of the workflow—turning dissent into infrastructure rather than leaving it dependent on user prompt quality, developer vigilance, or informal red-teaming.

7.2 Limitations

Correlated model failures. If all three roles are instantiated on the same foundation model, they share systematic biases, training-data blind spots, and failure modes. The protocol *reduces* but does not *eliminate* the risk of correlated failures. Mitigations include: (i) model diversification across roles (demonstrated in the pilot with Gemini and Kimi), (ii) authority grounding with external rule sets that explicitly enumerate known vulnerability classes, and (iii) periodic human audit of Ipcha Agent coverage.

Domain generality. Ipcha Mistabra is designed with domain-agnostic abstractions (Claim, Evidence, Finding, Gate Rule). However, all examples and evaluation targets in this paper are drawn from the security domain. Applicability to other domains (medical guideline review, legal contract analysis, scientific peer review) is plausible given the abstract formal structure but remains an untested hypothesis.

Ground-truth cost. Establishing ground truth for finding recall/precision requires expert panels, which are expensive and domain-specific.

Ipcha Score validity. As discussed in Section 4, the score is a proxy with known failure modes. The pilot backcheck revealed a concrete false-negative case: the v1.0.0→v1.1.0 paper revision resolved 9 findings yet produced $IS = 0.099$ (low band), because the changes were additive rather than substitutive. This underscores the recommendation to report IS alongside IS_w and finding-level metrics.

Pilot limitations. The pilot case study ($n = 1$) demonstrates protocol feasibility but does not constitute controlled evaluation. The cost-effectiveness observation ($0.20 for 12 findings) is compelling but requires comparison against manual expert review on the same artifact. Post-hardening re-verification (a second IM pass on the revised specification) was not performed.

8 Conclusion

Ipcha Mistabra institutionalizes contradiction as a verification primitive for safety-relevant text artifacts. We provide a formal specification with five definitions, pseudocode with specified subfunctions, and the Ipcha Score as a quantitative proxy for semantic displacement between review phases. A pilot case study on a production design specification demonstrated the protocol's ability to surface critical vulnerabilities—including injection attacks and circular LLM dependencies—that passed conventional review, at negligible cost.

We argue that institutionalized contradiction should be treated as a foundational systems primitive for AI workflows where decisions must survive scrutiny beyond surface-level fluency.

Future work includes (i) controlled evaluation across artifact classes (RQ1–RQ3), (ii) empirical calibration of the Ipcha Score against human expert ratings (RQ4), (iii) cross-domain validation (medical, legal, scientific review), (iv) robustness analysis against adversarial text inputs, and (v) model-diversification strategies to mitigate correlated failures.

Funding

No external funding.

Author Contributions

O. Baer: Conceptualization; Methodology; Software; Validation; Writing (Original Draft); Writing (Review & Editing).

Conflict of Interest

The author declares no conflict of interest.

Data Availability

The pilot data, reference implementation, and artifact manifest are available at `https://github.com/mrwind-up-bird/ipcha-mistabra`.

References

A Prompt Templates for LLM Instantiation

The following templates illustrate how the protocol subfunctions can be instantiated with an LLM. They are reference implementations; production deployments should adapt them to the target model and domain.

ExtractClaims Prompt Template

```
SYSTEM: You are a claim extraction agent. Given a text
artifact, extract every atomic, verifiable claim and every
unstated assumption required for that claim to hold.

OUTPUT FORMAT (JSON array):
[
  {
    "id": "C001",
    "claim": "<atomic statement>",
    "source_sentence": "<original text>",
    "assumptions": [
      {"id": "A001", "text": "<unstated prerequisite>"}
    ]
  }
]

Rules:
```

```
- One verifiable proposition per claim (atomic).
- Flag implicit prerequisites as assumptions.
- Do NOT include opinions, definitions, or tautologies.
```

IpchaContradict Prompt Template

```
SYSTEM: You are the Ipcha Agent (contradictor). Your role
is structured falsification. For each claim, construct the
strongest counter-hypothesis, check against authority docs,
and identify missing evidence.

INPUT: Claims JSON, Proponent output, Authority documents.

For each claim, produce:
{
  "claim_id": "C001",
  "counter_hypotheses": ["<strongest counter-narrative>"],
  "authority_violations": ["<norm/section violated>"],
  "missing_evidence": ["<what is needed to confirm>"],
  "severity": "critical|high|medium|low",
  "finding_status": "accepted|refuted|needs-evidence"
}

Rules:
- Assume the claim is FALSE until proven otherwise.
- Ground every objection in authority docs or logic.
- Do NOT generate contrarian noise without substance.
```

figures/ipcha_architecture_paper.pdf

Figure 1: High-level architecture of the Ipcha Mistabra protocol: reasoning layer (Proponent → Ipcha Agent → Auditor) with a verification and memory layer providing authority grounding, epistemic memory, and workflow persistence.

figures/ipcha_score_chart.pdf

Figure 2: Interpretation of the Ipcha Score. Thresholds are preliminary; empirical calibration is required. Higher values indicate stronger semantic displacement between Proponent and final validated output.